LAPORAN UTS

ALGORITMA PEMROGRAMAN LANJUT

Kelompok 10

Disusun oleh:

Anugerah Fakhriza Reswara (2509106025)

Muhammad Riyadh Zuhair (2509106029)

Nasha Nuron Wahdah (2509106037)

Kelas (A2‘25)

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

DAFTAR ISI

DAFTAR ISI **2**

BAB 1 PENDAHULUAN **1**

1.1 Latar Belakang 1

1.2 Rumusan Masalah 1

1.3 Tujuan Proyek 2

1.4 Batasan Masalah 2

1.5 Tautan Referensi Proyek 2

BAB 2 LANDASAN TEORI DAN TINJAUAN PUSTAKA **3**

2.1 Konsep Dasar Pemrograman C++ 3

2.2 Struktur Data Array 3

2.3 Teori Sepuluh Algoritma Pengurutan (Sorting) 4

2.4 Teori Empat Algoritma Pencarian (Searching) 5

BAB 3 PENJELASAN DAN IMPLEMENTASI PROGRAM **6**

3.1 Tema dan Konsep Program: "Sistem Administrasi Perbankan Digital" 6

3.2 Implementasi Detail Materi Pemrograman Lanjut 6

BAB 4 ARSITEKTUR DAN METODOLOGI EKSEKUSI **7**

4.1 Desain Arsitektur Sistem 7

4.2 Metodologi Pengolahan Data 8

4.2.1 Mekanisme Pemilihan Algoritma (Sorting) 8

4.2.2 Prosedur Pencarian (Searching) & Pre-kondisi Data 8

4.2.3 Visualisasi Data Multidimensi (Matriks) 8

4.2.4 Implementasi Pointer pada Manajemen Saldo 8

BAB 5 ANALISIS HASIL DAN KESIMPULAN **10**

5.1 Analisis Komparatif Performansi Algoritma Pengurutan 10

5.2 Evaluasi Strategi Pencarian Data 10

5.3 Pembahasan Struktur Data dan Modularitas 11

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Dalam era transformasi digital yang masif, sektor perbankan merupakan salah satu instansi yang paling bergantung pada pengolahan data dalam volume besar (Big Data). Setiap detik, ribuan transaksi terjadi, dan informasi nasabah harus dapat diakses, diurutkan, serta dicari dengan latensi yang seminimal mungkin. Ketidakmampuan sistem dalam mengelola data secara efisien tidak hanya berdampak pada penurunan performa aplikasi, tetapi juga pada tingkat kepercayaan nasabah terhadap integritas layanan perbankan.

Permasalahan utama dalam pengembangan perangkat lunak sering kali terletak pada pemilihan algoritma yang kurang tepat. Sebagai contoh, penggunaan algoritma pengurutan dengan kompleksitas $O(n^2)$ pada dataset jutaan nasabah akan memakan waktu komputasi yang tidak masuk akal. Oleh karena itu, penguasaan terhadap algoritma tingkat lanjut seperti sorting (pengurutan) dan searching (pencarian) menjadi kompetensi mutlak bagi seorang pengembang perangkat lunak. Proyek ini dirancang sebagai jembatan antara teori akademik dan implementasi praktis, di mana simulasi perbankan dijadikan studi kasus untuk menguji efektivitas berbagai metodologi pemrograman C++.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam proyek ini adalah:

1. Bagaimana mengimplementasikan struktur data heterogen menggunakan Struct dan manajemen memori melalui Pointer untuk merepresentasikan entitas nasabah?

2. Bagaimana efektivitas integrasi 10 algoritma pengurutan dan 4 algoritma pencarian dalam sebuah sistem yang bersifat modular?

3. Bagaimana representasi data transaksional dapat divisualisasikan secara efektif menggunakan struktur Array 2 Dimensi (Matriks)?

1.3 Tujuan Proyek

Adapun tujuan dari penyusunan proyek dan laporan ini adalah:

- **Tujuan Teoretis**: Memperdalam pemahaman mengenai kompleksitas algoritma (Time and Space Complexity) melalui implementasi langsung.
- **Tujuan Praktis**: Membangun aplikasi simulasi administrasi perbankan yang mampu melakukan manajemen data, pengurutan saldo nasabah, serta pencarian ID secara akurat.
- **Tujuan Akademik**: Mendemonstrasikan penguasaan materi perkuliahan Algoritma dan Pemrograman Lanjut, khususnya penggunaan fungsi, prototipe, array multidimensi, struct, dan pointer.

1.4 Batasan Masalah

Laporan ini membatasi pembahasan pada implementasi pemrograman prosedural menggunakan bahasa C++. Data nasabah yang digunakan bersifat simulatif (dummy data) dengan jumlah maksimal yang ditentukan oleh konstanta program. Sistem tidak menggunakan basis data eksternal, melainkan memanfaatkan memori utama (RAM) selama durasi eksekusi program.

1.5 Tautan Referensi Proyek

Untuk memberikan gambaran visual dan akses terhadap kode sumber proyek, berikut adalah tautan pendukung:

- **Video Penjelasan Materi**: [Link YouTube/Drive Anda Disini] (Berisi presentasi teknis mengenai algoritma yang diimplementasikan).
- **Repositori GitHub**: [Link GitHub Anda Disini] (Berisi kode sumber lengkap .cpp dan dokumentasi README).

BAB 2
LANDASAN TEORI DAN TINJAUAN PUSTAKA

2.1 Konsep Dasar Pemrograman C++

Prosedur dan Fungsi: Merupakan sub-program untuk menjalankan tugas spesifik. Prosedur (void) digunakan untuk aksi tanpa pengembalian nilai (seperti mencetak header), sementara fungsi digunakan untuk kalkulasi yang mengembalikan nilai (seperti menghitung rata-rata saldo). Modularitas ini memudahkan proses debugging dan maintenance.

Prototipe Fungsi: Deklarasi awal fungsi yang memberitahukan kompilator tentang tipe data kembalian dan parameter sebelum fungsi tersebut didefinisikan secara lengkap. Ini memungkinkan fungsi main() diletakkan di bagian atas kode untuk keterbacaan yang lebih baik.

Pointer: Variabel yang menyimpan alamat memori. Dalam manipulasi data nasabah yang besar, pointer memungkinkan akses langsung ke memori tanpa perlu menyalin seluruh data ke memori baru (pass by reference), sehingga meningkatkan kecepatan eksekusi program.

Struct: Tipe data komposit yang memungkinkan penggabungan variabel dengan tipe data berbeda (string, int, double) menjadi satu unit data "Nasabah". Struct berfungsi sebagai cetak biru (blueprint) objek dalam pemrograman prosedural.

2.2 Struktur Data Array

Array 1 Dimensi: Struktur data linear yang menyimpan elemen secara berurutan dalam memori yang berdekatan. Digunakan sebagai kontainer utama untuk menyimpan kumpulan objek nasabah secara terindeks.

Array 2 Dimensi (Matriks): Struktur data berbentuk baris dan kolom. Dalam konteks laporan ini, array 2D digunakan untuk merepresentasikan data transaksional bulanan, di mana baris mewakili ID nasabah dan kolom mewakili periode waktu transaksi (bulan/kuartal).

2.3 Teori Sepuluh Algoritma Pengurutan (Sorting)

- **Bubble Sort**: Algoritma berbasis pertukaran yang membandingkan dua elemen bersebelahan. Elemen "mengapung" ke posisi yang benar melalui iterasi berulang. Memiliki kompleksitas $O(n^2)$.
- **Selection Sort**: Bekerja dengan mencari elemen minimum/maksimum dari bagian yang belum terurut dan menukarnya ke posisi awal yang tepat. Sangat stabil namun kurang efisien untuk dataset besar.
- **Insertion Sort**: Cara kerjanya mirip dengan menyusun kartu di tangan; mengambil elemen satu per satu dan menyisipkannya ke posisi yang benar di antara elemen yang sudah terurut sebelumnya.
- **Shell Sort**: Merupakan improvisasi dari Insertion Sort. Algoritma ini membandingkan elemen dengan jarak (gap) tertentu, yang secara bertahap diperkecil hingga menjadi satu, sehingga mempercepat proses pengurutan data yang letaknya berjauhan.
- **Quick Sort**: Algoritma Divide and Conquer yang menggunakan pivot untuk membagi array menjadi dua sub-array. Efisiensi rata-ratanya mencapai $O(n \log n)$, menjadikannya salah satu yang tercepat secara praktis.
- **Merge Sort**: Algoritma rekursif yang membagi array menjadi bagian-bagian terkecil, mengurutkannya, lalu menggabungkannya kembali (merging) secara teratur. Menjamin stabilitas urutan data.
- **Heap Sort**: Menggunakan struktur data Binary Heap (pohon biner lengkap) untuk menemukan nilai terbesar secara berulang dan memindahkannya ke akhir array.
- **Radix Sort**: Algoritma pengurutan non-perbandingan yang mengelompokkan data berdasarkan digit angka dari posisi terkecil ke terbesar (satuan, puluhan, dst.).
- **Exchange Sort**: Mirip dengan Bubble Sort namun secara sistematis membandingkan elemen pertama dengan seluruh elemen sesudahnya sebelum beralih ke elemen kedua, untuk memastikan posisi elemen jangkar sudah tepat.
- **Tree Sort**: Proses pengurutan yang melibatkan pembangunan Binary Search Tree (BST). Data dimasukkan ke dalam pohon, kemudian dibaca kembali menggunakan metode In-order Traversal untuk menghasilkan urutan yang teratur.

2.4 Teori Empat Algoritma Pencarian (Searching)

- **Sequential Search**: Metode pencarian paling mendasar dengan memeriksa setiap elemen satu per satu dari indeks awal hingga akhir. Cocok untuk data acak atau tidak terurut.

- **Binary Search**: Pencarian efisien pada data yang sudah terurut. Algoritma ini membagi wilayah pencarian menjadi dua di setiap langkahnya, sehingga mengurangi waktu pencarian secara drastis menjadi $O(\log n)$.

- **Interpolation Search**: Pengembangan dari Binary Search untuk data dengan sebaran nilai yang merata. Menggunakan rumus posisi estimasi (mirip mencari kata di kamus berdasarkan huruf depan) untuk menemukan target lebih cepat.

- **Fibonacci Search**: Metode pencarian pada array terurut menggunakan deret Fibonacci sebagai titik pemisah rentang. Kelebihannya adalah hanya melibatkan operasi penambahan dan pengurangan sederhana, tanpa perkalian atau pembagian kompleks.

BAB 3
PENJELASAN DAN IMPLEMENTASI PROGRAM

3.1 Tema dan Konsep Program: "Sistem Administrasi Perbankan Digital"

Program ini mengadopsi skenario dunia nyata di mana sebuah institusi perbankan memerlukan alat internal untuk mengelola database nasabah. Konsep operasionalnya meliputi:

- Manajemen Identitas: Setiap nasabah memiliki ID unik, Nomor Rekening, Nama, Jenis Akun (Tabungan/Giro/Deposito), dan Saldo.
- Analisis Keuangan: Kemampuan untuk mengurutkan nasabah berdasarkan saldo guna mengidentifikasi nasabah High Net Worth (prioritas).
- Verifikasi Data: Kemampuan mencari nasabah secara instan melalui ID untuk mempermudah proses transaksi.

3.2 Implementasi Detail Materi Pemrograman Lanjut

Implementasi program ini merupakan kompilasi dari berbagai teknik pemrograman tingkat lanjut:

- **Struktur Data Teragregasi (Struct)**: Penggunaan struct Nasabah memungkinkan program untuk memperlakukan kumpulan variabel (string, int, double) sebagai satu objek tunggal. Hal ini mencerminkan prinsip enkapsulasi sederhana dalam pemrograman.
- **Manajemen Alamat Memori (Pointer)**: Pointer digunakan pada fungsi demoPointer() untuk menunjukkan bagaimana perubahan nilai saldo dapat dilakukan melalui alamat memori. Ini adalah teknik krusial dalam perbankan untuk memastikan sinkronisasi data tanpa redundansi.
- **Array Multidimensi (Matriks)**: Array 2D riwayatTransaksi[MAX_ROWS][MAX_COLS] digunakan untuk menyimpan data transaksi selama 4 bulan terakhir untuk 5 nasabah sampel. Ini mengilustrasikan hubungan antara entitas (baris) dan waktu (kolom).
- **Modularitas Kode (Functions & Prototypes)**: Kode dirancang sedemikian rupa sehingga setiap algoritma (seperti quickSort, binarySearch, dll) berdiri sebagai fungsi mandiri. Penggunaan prototipe fungsi di awal program menjamin struktur kode yang bersih dan profesional.

BAB 4
ARSITEKTUR DAN METODOLOGI EKSEKUSI

4.1 Desain Arsitektur Sistem

Sistem ini dibangun dengan arsitektur Top-Down Modular, di mana fungsi utama bertindak sebagai pengatur alur, sementara logika algoritma diisolasi dalam modul-modul independen. Berikut adalah dekomposisi arsitekturnya:

- **Global Data Layer**: Layer ini mendefinisikan struktur data fundamental seperti struct Nasabah dan array global daftarNasabah[]. Dengan menempatkan data pada level global (dengan kontrol akses tertentu), sistem memastikan bahwa seluruh fungsi sorting dan searching memiliki titik akses data yang seragam tanpa perlu mengirimkan salinan array yang besar antar fungsi, sehingga mengoptimalkan performa memori.

- **Initialization Layer**: Sebelum menu utama ditampilkan, program mengeksekusi fungsi isiDataAwal(). Fungsi ini berperan sebagai Data Seeder yang mengisi memori dengan dataset awal melalui teknik inisialisasi array. Hal ini bertujuan agar sistem langsung siap digunakan untuk pengujian tanpa memerlukan input manual yang repetitif setiap kali program dijalankan.

- **Core Engine Layer (Algoritma)**: Merupakan jantung dari program yang berisi 10 algoritma sorting dan 4 algoritma searching. Setiap algoritma diimplementasikan dalam fungsi terpisah. Arsitektur ini memungkinkan program untuk melakukan "perbandingan performa" secara paralel, di mana satu dataset yang sama dapat diuji oleh berbagai algoritma tanpa merusak dataset utama.

- **Security & Validation Layer**: Sistem menyertakan logika pengecekan sebelum eksekusi searching. Program secara otomatis mendeteksi apakah data dalam keadaan terurut atau tidak. Jika tidak, sistem akan memberikan notifikasi atau menjalankan sorting otomatis (Quick Sort) guna menjamin validitas hasil algoritma Binary/Interpolation search.

- **Presentation Layer (UI/UX)**: Menggunakan library iomanip dan sistem menu berbasis switch-case. Desain output diatur sedemikian rupa menggunakan manipulator teks (setw, right, left) untuk menghasilkan tampilan tabel yang profesional, memastikan data finansial mudah dibaca oleh operator sistem.

4.2 Metodologi Pengolahan Data

Metodologi eksekusi dalam program ini dirancang untuk memastikan integritas data dan validitas hasil pengolahan.

4.2.1 Mekanisme Pemilihan Algoritma (Sorting)

Pengguna diberikan kebebasan untuk memilih salah satu dari 10 metode pengurutan melalui menu interaktif. Untuk setiap metode, program akan mencatat kondisi awal array, menjalankan algoritma pengurutan pada atribut saldo, dan menampilkan hasilnya dalam bentuk tabel. Prosedur ini menggunakan teknik swapping objek struct secara utuh, sehingga seluruh atribut nasabah (ID, Nama, No. Rekening) ikut berpindah bersama saldonya tanpa merusak relasi data.

4.2.2 Prosedur Pencarian (Searching) & Pre-kondisi Data

Terdapat integrasi cerdas antara proses pencarian dan pengurutan. Karena algoritma seperti Binary Search dan Interpolation Search secara teoritis membutuhkan data yang terurut (sorted), program diimplementasikan dengan logika pengecekan. Jika pengguna mencoba menjalankan metode pencarian tersebut pada data acak, sistem akan memicu prosedur sorting otomatis (biasanya menggunakan Quick Sort) sebelum memulai proses pencarian. Hal ini dilakukan untuk mencegah kegagalan logika (logical error) pada hasil pencarian.

4.2.3 Visualisasi Data Multidimensi (Matriks)

Implementasi array 2 dimensi (matriks) dieksekusi melalui modul khusus yang memetakan saldo nasabah ke dalam periode waktu (bulan). Program melakukan iterasi pada baris untuk mewakili entitas nasabah dan kolom untuk mewakili riwayat transaksi. Metodologi ini menunjukkan bagaimana data transaksional yang kompleks dapat dikelola dalam struktur yang sistematis.

4.2.4 Implementasi Pointer pada Manajemen Saldo

Salah satu fitur krusial dalam program adalah penggunaan pointer untuk mendemonstrasikan

perubahan data langsung di memori. Dalam skenario perbankan, ini merepresentasikan aksi perubahan saldo instan. Dengan menggunakan operator dereference (*), program memanipulasi nilai saldo asli tanpa melakukan penyalinan data, yang secara teknis menghemat penggunaan overhead memori.

BAB 5
ANALISIS HASIL DAN KESIMPULAN

5.1 Analisis Komparatif Performansi Algoritma Pengurutan

Berdasarkan hasil pengujian running program pada dataset nasabah, didapatkan analisis sebagai berikut:

- **Efisiensi Waktu (Time Complexity)**: Algoritma Quick Sort dan Merge Sort menunjukkan efisiensi tertinggi pada dataset yang lebih besar karena menggunakan pendekatan rekursif Divide and Conquer. Sebaliknya, Bubble Sort dan Selection Sort memerlukan iterasi perbandingan yang jauh lebih banyak, yang ditandai dengan durasi eksekusi yang terasa lebih lambat secara mikroskopis saat menangani dataset mendekati batas MAX_NASABAH.

- **Karakteristik Penggunaan Memori (Space Complexity)**: Meskipun Merge Sort sangat stabil, ia memerlukan memori tambahan untuk proses penggabungan array sementara. Di sisi lain, Heap Sort dan Quick Sort lebih efisien dalam penggunaan memori karena melakukan pengurutan secara langsung pada array asal (in-place sorting).

- **Akurasi Radix Sort**: Pengujian pada ID nasabah menggunakan Radix Sort membuktikan bahwa pengurutan berbasis digit sangat efektif untuk data numerik bulat tanpa memerlukan komparasi nilai antar elemen, memberikan alternatif kecepatan yang signifikan untuk tipe data ID.

5.2 Evaluasi Strategi Pencarian Data

Analisis terhadap keempat metode pencarian memberikan gambaran efektivitas sebagai berikut:

- **Kecepatan Akses Terarah**: Interpolation Search terbukti menjadi yang tercepat dalam mencari saldo tertentu karena ia memprediksi posisi elemen target menggunakan rumus interpolasi linier. Namun, metode ini hanya efektif jika distribusi saldo nasabah relatif merata. Jika saldo memiliki rentang yang sangat timpang (ekstrim), performanya menurun mendekati pencarian sekuensial.

- **Stabilitas Binary Search**: Binary Search menunjukkan performa yang paling konsisten pada data terurut dengan membagi rentang pencarian secara logaritmik (O(log n)), menjadikannya standar yang paling handal dalam modul perbankan ini.

- **Fleksibilitas Sequential Search**: Meski secara teknis paling lambat, Sequential Search adalah satu-satunya algoritma yang dapat digunakan kapan saja tanpa perlu mengurutkan data terlebih dahulu. Ini sangat berguna untuk pencarian cepat pada dataset kecil yang baru saja dimasukkan secara acak.

5.3 Pembahasan Struktur Data dan Modularitas

Implementasi struct memberikan kemudahan luar biasa dalam manajemen kode. Tanpa struct, pengembang harus mengelola 5 array terpisah untuk setiap atribut nasabah, yang akan meningkatkan risiko inkonsistensi data saat proses sorting. Penggunaan fungsi prototipe juga membantu dalam mengisolasi logika algoritma. Jika satu algoritma (misalnya Shell Sort) memerlukan perbaikan, perubahan dapat dilakukan hanya pada fungsi tersebut tanpa mempengaruhi fungsi pencarian atau menu utama. Hal ini membuktikan bahwa arsitektur program yang modular sangat penting dalam skala sistem informasi perbankan yang terus berkembang.